

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Thu Sep 10 23:36:09 2026

@author: goot
"""

import numpy as np
import matplotlib.pyplot as plt
import math as math

#%%Problem 3
#Algorithm implementation
def expo(x):
    y = math.e**x
    return y -1

x = 9.999999995000000*(10**-10)
x = np.linspace(-.00000000000001,.00000000000001,100)
y = expo(x)
y1 = np.expm1(x)

plt.plot(x,y)
plt.plot(x,y1)
plt.show()

#%% Calculator
weex = 9.999999995*10**(-10)
nnumber = ((0.5*10**(-17))-1)/(weex)

funcf = lambda n: weex**((2*n+2)/(n+1))

n = np.linspace(0,100,101)
bruh = funcf(n) / (10**-9)

bing = x/1+x**2/2

#%% Problem 4c

def driver():
    # use routines
    f = lambda x: 2*x-1-np.sin(x)
    a = 0
    b = 1

```

```

tol = 0.5*10**(-9)
[astar,ier] = bisection(f,a,b,tol)
print('the approximate root is',astar)
print('the error message reads:',ier)
print('f(astar) =', f(astar))

f = lambda x: x**3+x-4
a = 1
b = 4

tol = 10**-3

[astar,ier] = bisection(f,a,b,tol)
print('the approximate root is',astar)
print('the error message reads:',ier)
print('f(astar) =', f(astar))
#%% Problem 5
# define routines
def bisection(f,a,b,tol):

#   Inputs:
#   f,a,b      - function and endpoints of initial interval
#   tol        - bisection stops when interval length < tol

#   Returns:
#   astar      - approximation of root
#   ier        - error message
#               - ier = 1 => Failed
#               - ier = 0 == success

#   first verify there is a root we can find in the interval

fa = f(a)
fb = f(b);
if (fa*fb>0):
    ier = 1
    astar = a
    return [astar, ier]

#   verify end points are not a root
if (fa == 0):
    astar = a
    ier = 0
    return [astar, ier]

if (fb == 0):
    astar = b
    ier = 0
    return [astar, ier]

count = 0

```

```

d = 0.5*(a+b)
while (abs(d-a)> tol):
    fd = f(d)
    if (fd ==0):
        astar = d
        ier = 0
        return [astar, ier]
    if (fa*fd<0):
        b = d
    else:
        a = d
        fa = fd
    d = 0.5*(a+b)
    count = count +1
#     print('abs(d-a) = ', abs(d-a))

    astar = d
    ier = 0
    print('count = ', count)
    return [astar, ier]

driver()

### Problem 6
#a - plotting
f = lambda x: x-4*np.sin(2*x)-3
x = np.linspace(-np.pi,3*np.pi,300)

# x = np.linspace(6,8,300)|
plt.axhline(y=0,color="k")
plt.title("Problem 6 Outcomes (There are 5 roots to this equation)")

plt.plot(x,f(x))
plt.grid("minor")

#b - fixed point stuff and roots
def drivin():
    # define routines
    #x = 3-4sin(x)
    f1 = lambda x: -np.sin(2*x)+5*x/4-3/4
    #fixed point is alpha2 = 3.09...

    Nmax = 100
    tol = 0.5*10**-10

    # test f1 '''
    x0 = [-1,-.5,1.5,3.0,4,7]
    for x in x0:

        [xstar1,ier] = fixedpt(f1,x,tol,Nmax)

```

```

    print('the approximate fixed point is:',xstar1)
    print('f1(xstar):',f1(xstar1))
    print('Error message reads:',ier)

def fixedpt(f,x0,tol,Nmax):

    ''' x0 = initial guess'''
    ''' Nmax = max number of iterations'''
    ''' tol = stopping tolerance'''

    count = 0
    xstar = np.zeros((Nmax+1,1))
    xstar[0] = x0
    while (count <Nmax):
        count = count +1
        x1 = f(x0)
        if (abs(x1-x0) <tol):
            xstar = x1
            ier = 0
            return [xstar,ier]
        x0 = x1
        xstar = x1

    ier = 1
    return [xstar, ier]
drivin()
print("All except the first root converge. The closeness of the first two roots t

```